

Лабораторная работа №1

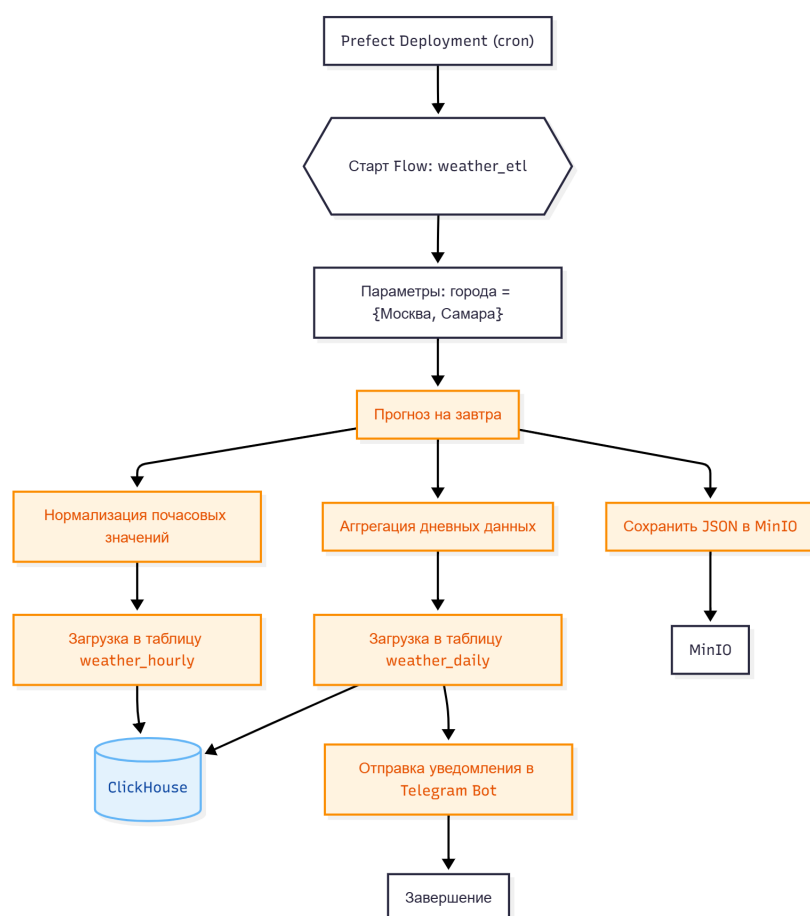
Построение ETL-пайплайна на Prefect:

1. Архитектура решения

Цель работы — реализовать полноценный ETL-пайплайн для ежедневного прогноза погоды на завтра для городов **Москва** и **Самара**, включающий:

- извлечение данных из Open-Meteo;
- сохранение сырых данных в MinIO;
- преобразование почасовых и дневных значений;
- загрузку нормализованных данных в ClickHouse;
- отправку итогового уведомления в Telegram.

Общая схема пайплайна:



Используемые инструменты:

- **Prefect 3.x** — оркестрация задач и управление пайплайном
- **Open-Meteo API** — источник метеоданных
- **MinIO** — объектное S3-хранилище для сохранения JSON
- **ClickHouse** — аналитическая СУБД для хранения таблиц

- **Docker Compose** — единый запуск инфраструктуры
- **Telegram Bot API** — отправка итогового прогноза пользователю

2. Источник данных

Используется публичный API Open-Meteo: <https://api.open-meteo.com/v1/forecast>

Основные параметры запроса:

- `latitude, longitude` — координаты города
- `hourly=temperature_2m,precipitation,wind_speed_10m,wind_direction_10m`
- `timezone=auto`
- `forecast_days=2` — используется только **завтрашний день**

Пример запроса для Москвы: [https://api.open-meteo.com/v1/forecast?](https://api.open-meteo.com/v1/forecast?latitude=55.75&longitude=37.62&hourly=temperature_2m,precipitation,wind_speed_10m,wind_direction_10m&timezone=auto&forecast_days=2)

`latitude=55.75&longitude=37.62&hourly=temperature_2m,precipitation,wind_speed_10m,wind_direction_10m&timezone=auto&forecast_days=2`

3. Extract → Transform → Load

3.1. Extract

Для каждого города выполняется:

- запрос прогноза на завтра;
- сохранение полученного JSON в MinIO по пути: `s3://weather-raw/raw//.json`

3.2. Transform

Преобразование включает два этапа.

а) Нормализация почасовых значений → таблица `weather_hourly`

Данные включают:

- название и ключ города;
- timestamp;
- температуру, осадки, скорость и направление ветра.

б) Агрегация дневных значений → таблица `weather_daily`

Рассчитываются:

- минимальная, максимальная и средняя температура;
- суммарные осадки;
- максимальная скорость ветра;
- флаги:
 - `strong_wind` — скорость ветра > 15 м/с
 - `heavy_precip` — осадки > 10 мм

3.3. Load

Создаются две таблицы ClickHouse.

Таблица `weather_hourly`:

```
city String,  
city_name String,  
ts DateTime,  
temperature Float32,  
precipitation Float32,  
wind_speed Float32,  
wind_direction Float32,  
date Date
```

Таблица `weather_daily`:

```
city String,  
city_name String,  
date Date,  
temp_min Float32,  
temp_max Float32,  
temp_avg Float32,  
precip_sum Float32,  
max_wind_speed Float32,  
strong_wind UInt8,  
heavy_precip UInt8
```

Загрузка выполняется через библиотеку `clickhouse-driver`.

4. Пайплайн Prefect

Flow состоит из четырёх задач:

```
extract_task --> transform_task --> load_task --> notify_task
```

Каждая задача завершается статусом `Completed()`.

5. Качество данных и обработка ошибок

Реализованы проверки:

- отсутствие сырого JSON → пропуск города;
- проверка наличия всех полей API;

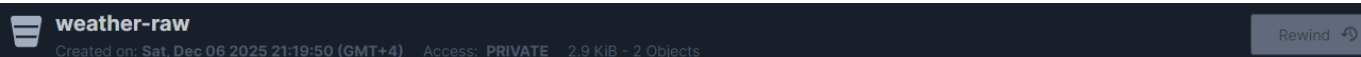
- согласованность длины почасовых массивов;
- автоматическое создание бакета MinIO при ошибке NoSuchBucket;
- обработка сетевых ошибок (requests.exceptions);
- проверка токена Telegram перед отправкой сообщения.

Потенциальные точки сбоя:

- недоступность Open-Meteo;
- неверные учётные данные Telegram;
- проблемы записи в ClickHouse;
- изменение структуры API.

6. Результаты работы

6.1 Содержимое MinIO



weather-raw

Created on: Sat, Dec 06 2025 21:19:50 (GMT+4) Access: PRIVATE 2.9 KiB - 2 Objects

weather-raw / raw / moscow / 2025-12-07.json

Create new path

Name	Last Modified	Size
2025-12-07.json	Today, 21:22	1.5 KiB

6.2 Данные в ClickHouse

```
from clickhouse_driver import Client

client = Client(host="clickhouse", port=9000,
                user="user", password="password",
                database="weather")

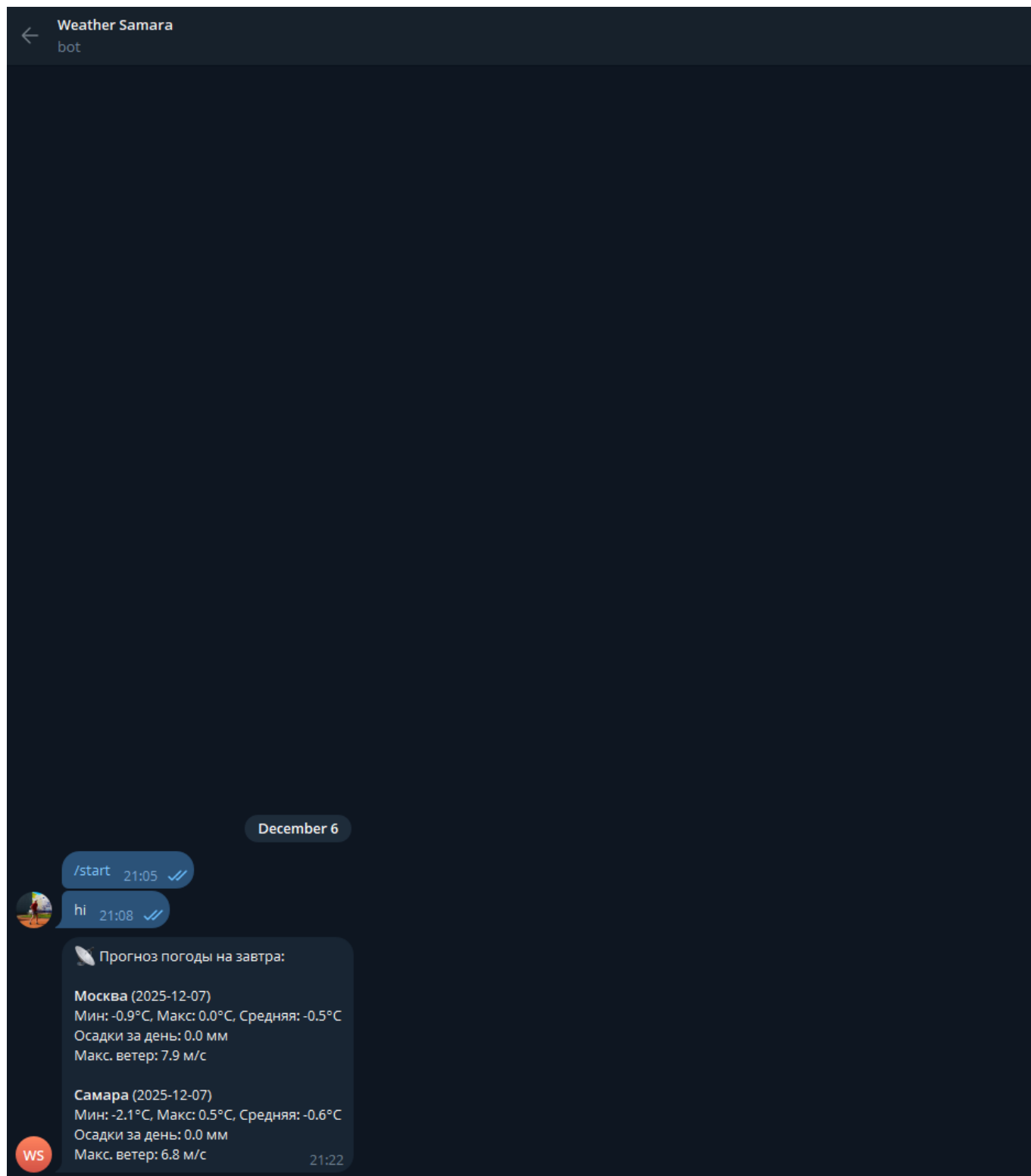
print("weather_hourly (sample)")
rows = client.execute("SELECT city, ts, temperature, precipitation FROM weather_hourly ORDER BY ts LIMIT 5")
for r in rows:
    print(r)

print("\n weather_daily (sample)")
rows = client.execute("SELECT city, date, temp_min, temp_max, temp_avg, precip_sum, max_wind_speed FROM weather_daily ORDER BY date, city")
for r in rows:
    print(r)

PY
weather_hourly (sample)
('samara', datetime.datetime(2025, 12, 7, 0, 0), -1.100000023841858, 0.0)
('moscow', datetime.datetime(2025, 12, 7, 0, 0), 0.0, 0.0)
('samara', datetime.datetime(2025, 12, 7, 0, 0), -1.100000023841858, 0.0)
('moscow', datetime.datetime(2025, 12, 7, 0, 0), 0.0, 0.0)
('samara', datetime.datetime(2025, 12, 7, 1, 0), -1.2999999523162842, 0.0)

weather_daily (sample)
('moscow', datetime.date(2025, 12, 7), -0.8999999761581421, 0.0, -0.44999998807907104, 0.0, 7.900000095367432)
('moscow', datetime.date(2025, 12, 7), -0.8999999761581421, 0.0, -0.44999998807907104, 0.0, 7.900000095367432)
('samara', datetime.date(2025, 12, 7), -2.0999999046325684, 0.5, -0.625, 0.0, 6.800000190734863)
('samara', datetime.date(2025, 12, 7), -2.0999999046325684, 0.5, -0.625, 0.0, 6.800000190734863)
root@7f82bd1c4a77:/app#
```

6.3. Telegram-уведомление



7. Выводы

В ходе работы был реализован полный ETL-конвейер:

- получение прогноза погоды из API;
- сохранение сырых данных в объектном хранилище;
- нормализация почасовых значений;
- вычисление дневной статистики;

- загрузка данных в ClickHouse;
- отправка уведомления пользователю;
- оркестрация процессов средствами Prefect.

Сложности возникли с корректным преобразованием временных меток и интеграцией ClickHouse. В качестве возможных улучшений можно выделить:

- автоматический деплой Prefect (Work Pool + Schedule),
- подключение мониторинга (Grafana + ClickHouse),
- расширение набора метеопоказателей,
- уведомления о погодных аномалиях.

Работа полностью соответствует требованиям задания.